

Style et mise en page avec Flexbox

Table des matières

I. Contexte	3
II. Le composant StyleSheet de React Native	3
III. Exercice : Appliquez la notion	5
IV. La mise en page avec Flexbox	6
V. Exercice : Appliquez la notion	17
VI. Essentiel	18
VII. Auto-évaluation	18
A. Exercice final	18
B. Exercice : Défi	20
Contenus annexes	21
Solutions des exercices	34

I. Contexte

Durée : 1 h

Environnement de travail : Une application React Native initialisée ou utiliser <https://snack.expo.io/>

Pré-requis : Les props et le state, les composants React Native

Contexte

Nous savons désormais utiliser les props, le state, et plus globalement les composants et leur composition. Cependant, pour l'instant, ce n'est pas très « design », c'est même assez brut de décoffrage !

Cela tombe bien, ce module a pour objectif de nous apprendre à rajouter une touche de design à nos composants afin de pouvoir réaliser des interfaces utilisateurs qui donnent envie d'être utilisées.

Et pour cela, React Native est vraiment très pratique, car il se base sur les technologies du Web qui sont connues pour être de supers outils de mise en page.

Pour la plupart très proches de CSS, les différentes propriétés que nous allons utiliser vont nous permettre de faire varier la taille de nos textes, de colorer nos éléments ou encore de positionner ces derniers les uns par rapport aux autres.

II. Le composant StyleSheet de React Native

Objectifs

- Prendre en main le design d'une application React Native
- S'exercer à un premier affichage de composants graphiques

Mise en situation

React Native utilise une technologie sous-jacente nommée Yoga¹ et développée par Facebook, qui permet de mimer le fonctionnement de la spécification FlexBox du Web. Le rendu graphique des composants en React Native se configure en utilisant la propriété `style` qu'ils proposent. Celle-ci attend un objet auquel il est possible d'affecter de nombreuses propriétés proches de celles existant en CSS. Toutes les propriétés utilisables sur les différents composants sont disponibles dans la documentation².

Pour le confort et la vérification des propriétés que nous utilisons, React Native met à disposition une API qui va procéder à un certain nombre de vérifications avant de générer notre objet de design final. Cette API se nomme `StyleSheet` et s'utilise très facilement.

Exemple

Imaginons que nous souhaitons centrer un texte « *Bienvenue* » écrit en blanc et en gras. On souhaite aussi spécifier une taille de titre et changer l'arrière-plan de l'application en bleu.

```
1 import * as React from 'react';
2 import { Text, View, StyleSheet } from 'react-native';
3
4 export default function App() {
```

¹ <https://yogalayout.com/>

² <https://reactnative.dev/docs/layout-props>

```

5   return (
6     <View style={styles.container}>
7       <Text style={styles.title}>Bienvenue</Text>
8     </View>
9   );
10 }
11
12 const styles = StyleSheet.create({
13   container: {
14     backgroundColor: 'blue',
15     flex: 1,
16     justifyContent: 'center',
17     alignItems: 'center',
18   },
19   title: {
20     color: 'white',
21     fontSize: 26,
22     fontWeight: 'bold',
23   },
24 });

```

On peut observer qu'on définit un objet `styles` dans le fichier, généralement en bas de page, mais on pourrait également le mettre dans un autre fichier.

Dans cet objet, chaque clé nommée arbitrairement (`container` et `title` ici) représente un morceau de design que l'on va pouvoir greffer sur un élément de notre interface.

Et c'est ce que l'on fait en passant la props `style` avec le bon objet à notre `View` : `<View style={styles.container}>`.

- Notre conteneur définit donc une couleur de fond bleu et utilise d'autres propriétés Flex que nous verrons un peu plus loin
- Le texte, lui, définit sa couleur en blanc, sa taille à 26 pixels et se définit comme gras

Des styles dynamiques

Jusqu'ici, nos styles étaient figés et ne pouvaient pas changer en fonction de l'état de l'application. Nous allons voir qu'il est en fait possible de faire évoluer le style de l'application.

Dans l'exemple ci-dessous, lorsque l'on clique sur un bouton, on fait varier la couleur de l'arrière-plan tout en gardant les autres propriétés. Pour cela, on passe un tableau à la props `style` du composant `View`, qui s'occupera de fusionner les différentes propriétés en écrasant les premières avec les dernières du tableau.

Exemple

```

1 import * as React from 'react';
2 import { Button, Text, View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   const [bgColor, setBgColor] = React.useState('blue');
6
7   return (
8     <View
9       style=[
10         styles.container,
11         {
12           backgroundColor: bgColor,

```

```

13     },
14   ]}>
15   <Button title="Rouge" onPress={() => setBgColor('red')} />
16   <Text style={styles.title}>Bienvenue</Text>
17   <Button title="Vert" onPress={() => setBgColor('green')} />
18 </View>
19 );
20 }
21
22 const styles = StyleSheet.create({
23   container: {
24     flex: 1,
25     justifyContent: 'center',
26     alignItems: 'center',
27   },
28   title: {
29     color: 'white',
30     fontSize: 26,
31     fontWeight: 'bold',
32   },
33 });

```

Quand on clique sur les boutons, la couleur change sur l'interface. Voilà, notre application devient dynamique !

Dans cet exemple, on injecte du style en se basant sur le state, mais on pourrait très bien utiliser une props ou n'importe quelle autre variable.

Syntaxe À retenir

- Utiliser du style avec les composants React Native est très simple et s'inspire grandement de la très populaire syntaxe CSS.
- `StyleSheet` est une API mise à disposition pour s'assurer que l'on écrit du style correct et qui, si l'IDE le permet, ajoute de l'auto-complétion, ce qui est très pratique.
- Enfin, on peut faire varier les styles dynamiquement en fonction de variables comme le state ou les props, ce qui nous permettra plus tard, nous le verrons, de gérer des animations.

Complément

- <https://reactnative.dev/docs/view-style-props>
- <https://reactnative.dev/docs/style>

III. Exercice : Appliquez la notion

Question

En s'inspirant du code de l'exemple vu précédemment (*voir ci-dessous*) et en utilisant la documentation liée sur les props, essayez d'afficher 3 textes (rouge, blanc et bleu) avec un fond d'application qui change toutes les 5 secondes du jaune au vert, et vice-versa (en utilisant `setInterval`).

```

1 import * as React from 'react';
2 import { Button, Text, View, StyleSheet } from 'react-native';
3
4 export default function App() {

```

```

5  const [bgColor, setBgColor] = React.useState('blue');
6
7  return (
8    <View
9      style={[
10        styles.container,
11        {
12          backgroundColor: bgColor,
13        },
14      ]}>
15      <Button title="Rouge" onPress={() => setBgColor('red')} />
16      <Text style={styles.title}>Bienvenue</Text>
17      <Button title="Vert" onPress={() => setBgColor('green')} />
18    </View>
19  );
20 }
21
22 const styles = StyleSheet.create({
23   container: {
24     flex: 1,
25     justifyContent: 'center',
26     alignItems: 'center',
27   },
28   title: {
29     color: 'white',
30     fontSize: 26,
31     fontWeight: 'bold',
32   },
33 });

```

IV. La mise en page avec Flexbox

Objectifs

- Comprendre les principes de base de la mise en page avec Flexbox
- Comprendre les différences avec la spécification web de Flexbox
- S'essayer à la mise en page React Native

Mise en situation

Tout comme nous l'avons vu précédemment, React Native utilise le moteur YogaLayout de Facebook pour contrôler la mise en page de l'application. Et, bien que très proche de la spécification web de Flexbox, le moteur présente quelques différences mineures avec cette dernière. Néanmoins, si l'on maîtrise la syntaxe web de Flexbox, il n'y aura aucune difficulté à appliquer ces principes à la mise en page d'une application React Native.

Conseil

Une description du fonctionnement de Flexbox peut être trouvée *ici* (cf. p.21).

Qu'est-ce que Flexbox ?

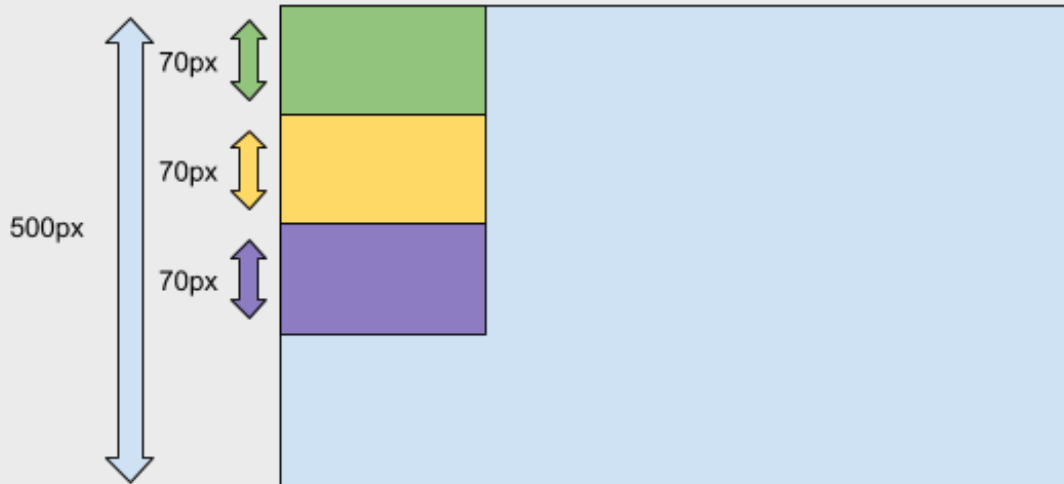
Flexbox est une spécification qui permet de répartir des composants dans une surface définie. Chaque élément possède une taille qui est définie manuellement ou qui dépend de son contenu. Ces éléments sont positionnés dans des conteneurs flexibles (en React Native, un conteneur est flexible par défaut) et Flexbox nous permet d'indiquer comment les répartir à l'intérieur de cette surface.

Le principe de base de Flexbox est l'utilisation d'un axe principal et d'un axe secondaire. Sur le Web, l'axe principal est l'axe horizontal, et le secondaire est le vertical. Cependant, sur React Native (généralement utilisé sur de petits écrans), les développeurs ont surchargé ce comportement pour mettre l'axe vertical en axe par défaut.

C'est pourquoi, si aucune modification n'est apportée par du style, le fait d'afficher trois boîtes à la suite les unes des autres avec un code React Native provoquera l'affichage suivant :

Exemple

```
1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     backgroundColor: 'lightblue',
17     flex: 1, // On met flex: 1 pour que l'arrière plan prenne toute la place disponible et pas
18     // seulement la taille des contenants
19   },
20   firstBox: {
21     backgroundColor: 'green',
22     height: 70,
23     width: 100,
24   },
25   secondBox: {
26     backgroundColor: 'yellow',
27     height: 70,
28     width: 100,
29   },
30   thirdBox: {
31     backgroundColor: 'purple',
32     height: 70,
33     width: 100,
34   },
35 });
```



Les trois boîtes sont affichées verticalement dans le coin supérieur gauche.

Répartir l'espace du parent

Les propriétés que nous allons voir se distinguent en deux catégories : les propriétés qui s'utilisent sur le parent (le conteneur flexible), et celles qui s'utilisent sur les éléments enfants.

Fondamental Différents types de propriétés

- **Pour les propriétés du parent** : elles servent à contrôler l'attribution de l'espace non occupé quand les éléments ne remplissent pas la boîte.
- **Pour les propriétés d'un enfant** : elles servent à définir comment se comporte un enfant par rapport aux autres dans la zone du parent, notamment lorsqu'il reste de l'espace non alloué.

Les principales propriétés de l'élément parent

La propriété `flexDirection` permet d'indiquer l'axe principal de positionnement des enfants de l'élément en prenant les valeurs `row` pour un axe horizontal, ou `column` pour un axe vertical. De plus, il est possible d'inverser le flux naturel d'affichage avec les propriétés `row-reverse` (les enfants se positionnent de droite à gauche) ou `column-reverse` (ils se répartissent de bas en haut).

La valeur par défaut en React Native est `column`.

Exemple

```
1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
}
```



```

12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     flexDirection: 'row'
17   },
18   firstBox: {
19     backgroundColor: 'green',
20     height: 70,
21     width: 100,
22   },
23   secondBox: {
24     backgroundColor: 'yellow',
25     height: 70,
26     width: 100,
27   },
28   thirdBox: {
29     backgroundColor: 'purple',
30     height: 70,
31     width: 100,
32   },
33 });

```



L'utilisation de la propriété `flexDirection` avec la valeur `row` au niveau de l'élément parent positionne immédiatement les éléments enfants sur un axe horizontal.

Exemple

```

1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     flexDirection: 'row-reverse'
17   },
18   firstBox: {
19     backgroundColor: 'green',
20     height: 70,
21     width: 100,
22   },
23   secondBox: {
24     backgroundColor: 'yellow',
25     height: 70,

```

```

26     width: 100,
27   },
28   thirdBox: {
29     backgroundColor: 'purple',
30     height: 70,
31     width: 100,
32   },
33 });
    
```



L'utilisation de la propriété `flexDirection` avec la valeur `row-reverse` au niveau de l'élément parent positionne immédiatement les éléments enfants sur un axe horizontal de droite à gauche.

La propriété `justifyContent` permet de faire varier la répartition des éléments le long de l'axe principal. Cette propriété peut prendre les valeurs `flex-start`, `center`, `flex-end`, `space-between`, `space-around` et `space-evenly`.

Par défaut, `justifyContent` prend la valeur `flex-start`, pour laquelle la répartition des éléments commence au début de l'axe principal.

Exemple

```

1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     justifyContent: 'center',
17     backgroundColor: 'grey',
18     height: 300,
19   },
20   firstBox: {
21     backgroundColor: 'green',
22     height: 70,
23     width: 100,
24   },
25   secondBox: {
26     backgroundColor: 'yellow',
27     height: 70,
28     width: 100,
29   },
30   thirdBox: {
31     backgroundColor: 'purple',
32     height: 70,
    
```

```

33   width: 100,
34 },
35 });

```



L'utilisation de la propriété `justifyContent` avec la valeur `center` au niveau de l'élément parent centre les éléments enfants sur l'axe principal (vertical, ici).

Exemple

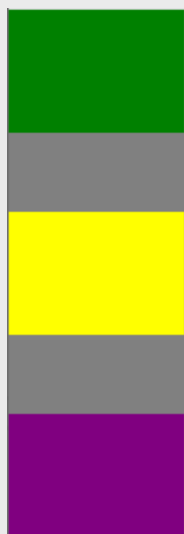
```

1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     justifyContent: 'space-between',
17     backgroundColor: 'grey',
18     height: 373,
19   },
20   firstBox: {
21     backgroundColor: 'green',
22     height: 70,
23     width: 100,
24   },
25   secondBox: {
26     backgroundColor: 'yellow',
27     height: 70,
28     width: 100,
29   },
30   thirdBox: {
31     backgroundColor: 'purple',
32     height: 70,

```

```

33     width: 100,
34   },
35 });
    
```



L'utilisation de la propriété `justifyContent` avec la valeur `space-between` au niveau de l'élément parent répartit les éléments enfants sur l'axe principal, en les séparant par un espace identique.

La propriété `alignItem` permet de faire varier la répartition des éléments le long de l'axe secondaire. Il peut prendre les valeurs `flex-start`, `flex-end`, `center`, `stretch` et `baseline`.

Par défaut, `alignItem` prend la valeur `stretch`, pour laquelle les éléments enfants sont étirés sur l'axe secondaire pour occuper tout l'espace disponible.

Exemple

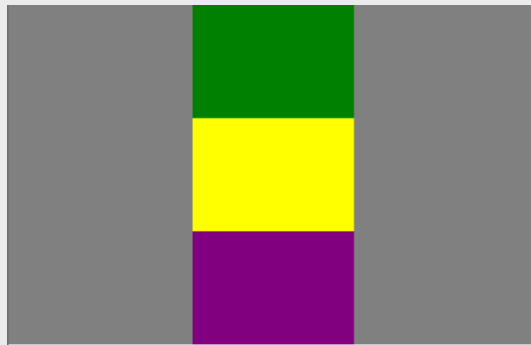
```

1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     alignItems: 'center',
17     backgroundColor: 'grey',
18   },
19   firstBox: {
20     backgroundColor: 'green',
21     height: 70,
22     width: 100,
23   },
24   secondBox: {
25     backgroundColor: 'yellow',
26     height: 70,
    
```

```

27   width: 100,
28 },
29 thirdBox: {
30   backgroundColor: 'purple',
31   height: 70,
32   width: 100,
33 },
34 });

```



L'utilisation de la propriété `alignItems` avec la valeur `center` au niveau de l'élément parent centre les éléments enfants sur l'axe secondaire (horizontal, ici).

Exemple

```

1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     alignItems: 'flex-end',
17     backgroundColor: 'grey',
18   },
19   firstBox: {
20     backgroundColor: 'green',
21     height: 70,
22     width: 100,
23   },
24   secondBox: {
25     backgroundColor: 'yellow',
26     height: 70,
27     width: 100,
28   },
29   thirdBox: {
30     backgroundColor: 'purple',
31     height: 70,
32     width: 100,

```

```
33 },
34 });
```



L'utilisation de la propriété `alignItems` avec la valeur `flex-end` au niveau de l'élément parent positionne les éléments enfants à la fin de l'axe secondaire (horizontal, ici).

Les principales propriétés des éléments enfants

La propriété `flexGrow` permet de définir un coefficient de croissance d'un élément enfant en fonction de l'espace total et des différentes valeurs de `flexGrow` des autres enfants d'un conteneur. `flexGrow` accepte un nombre positif, avec 0 comme valeur par défaut.

Le calcul de la taille finale d'un élément enfant se fait de la manière suivante : $\{\text{flexGrow de l'élément}\} * 1 / \{\text{somme des flexGrow de tous les enfants}\}$.

Exemple

```
1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     flexDirection: 'row',
17   },
18   firstBox: {
19     backgroundColor: 'green',
20     flexGrow : 2,
21     height: 70
22   },
23   secondBox: {
24     backgroundColor: 'yellow',
25     flexGrow : 1,
26     height: 70
27   },
28   thirdBox: {
29     backgroundColor: 'purple',
```

```

30   flexGrow : 1,
31   height: 70
32 },
33 });

```

L'élément conteneur a trois éléments enfants : l'élément `firstBox` a un `flexGrow` de 2, tandis que les éléments suivants ont un `flexGrow` de 1.



Cela signifie que l'expansion de l'élément `firstBox` doit correspondre à $2 * \frac{1}{4}$ de l'espace disponible, tandis que les deux autres enfants doivent chacun en occuper $\frac{1}{4}$. Le résultat final montre bien que le premier élément occupe 50 % de la taille du conteneur, tandis que les deux autres en occupent 25 % chacun.

La propriété `flexShrink` permet de définir dans quelles proportions la taille d'un élément enfant se réduit quand le conteneur est trop petit pour contenir tous ses enfants. `flexShrink` accepte un nombre positif, avec 1 comme valeur par défaut.

De la même manière que pour `flexGrow`, ce coefficient de réduction s'applique en fonction de la valeur de `flexShrink` de chacun des enfants du conteneur.

Le calcul de la réduction de la taille d'un élément enfant se fait de la manière suivante : **$\{\text{flexShrink de l'élément}\} * \frac{1}{\{\text{somme des flexGrow de tous les enfants}\}} * \{\text{espace manquant}\}$** .

Exemple

```

1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     flexDirection: 'row',
17     width: 250
18   },
19   firstBox: {
20     backgroundColor: 'green',
21     flexShrink : 3,
22     height: 70,
23     width: 100
24   },
25   secondBox: {
26     backgroundColor: 'yellow',
27     flexShrink : 1,
28     height: 70,
29     width: 100

```

```

30 },
31 thirdBox: {
32   backgroundColor: 'purple',
33   flexShrink : 1,
34   height: 70,
35   width: 100
36 },
37 });
    
```



L'élément conteneur d'une largeur de 250 unités doit inclure trois éléments de 100 unités : il est donc trop petit de 50 unités et les éléments enfants doivent être rétrécis en conséquence.

Cette réduction se calcule en fonction des valeurs de `flexShrink` de chacun des enfants, c'est-à-dire $(3 * 1/5) * 50 = 30$ pour l'élément `firstBox`, et $1/5 * 50 = 10$ pour les deux autres.

La propriété `alignSelf` permet pour un élément de surcharger spécifiquement la valeur de `alignItems` définie sur le parent.

Cette propriété peut prendre les valeurs suivantes : `auto`, `flex-start`, `flex-end`, `center`, `stretch`, `baseline`. La valeur par défaut de `alignSelf` est `auto`.

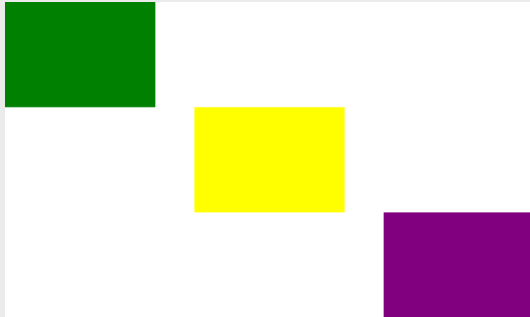
Exemple

```

1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     alignItems: 'center'
17   },
18   firstBox: {
19     backgroundColor: 'green',
20     height: 70,
21     width: 100,
22     alignSelf: 'flex-start'
23   },
24   secondBox: {
25     backgroundColor: 'yellow',
26     height: 70,
27     width: 100
28   },
29   thirdBox: {
30     backgroundColor: 'purple',
    
```



```
31   height: 70,  
32   width: 100,  
33   alignSelf: 'flex-end'  
34 },  
35 });
```



La propriété `alignItems` de l'élément parent définit que les éléments enfants doivent être centrés sur l'axe horizontal. Ce comportement est écrasé par la valeur de la propriété `alignSelf` des éléments enfants : ainsi, l'élément `firstBox` est positionné au début de l'axe horizontal, et l'élément `thirdBox` est positionné à la fin de l'axe horizontal.

Syntaxe À retenir

- React Native utilise YogaLayout, une librairie native qui va permettre d'utiliser une syntaxe très proche de la spécification CSS FlexBox. On peut donc réaliser très simplement des interfaces graphiques sur-mesure.
- En effet, les différences sont minimes et ne concernent que certaines propriétés, comme la valeur par défaut de `flexDirection` qui utilise le mode `column` par défaut.

Complément

- <https://reactnative.dev/docs/flexbox>
- <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- <https://flexboxfroggy.com/#fr>^{1 2}

V. Exercice : Appliquez la notion

Question

Vous allez créer trois boîtes avec une hauteur de 50px, mais sans largeur définie. Chacune devra avoir une couleur différente pour qu'on puisse la différencier des autres.

Il faut afficher ces boîtes côte à côte sur une ligne horizontale : la première doit prendre 2/6 de l'espace disponible, la seconde 3/6 de l'espace disponible et la dernière 1/6 de l'espace disponible.

¹ <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

² <https://flexboxfroggy.com/#fr>

VI. Essentiel

React Native permet très simplement de faire une mise en page professionnelle en se basant sur une syntaxe proche du CSS pour ses propriétés de mise en page. Son principal atout est le moteur Facebook YogaLayout, qui permet d'utiliser le système de mise en page FlexBox avec seulement quelques différences mineures comparé à la spécification web de Flexbox. La mise en page est donc très intuitive, facile et responsive.

Il fournit des outils très pratiques qui permettent de vérifier qu'on écrit les bonnes propriétés, comme l'API `StyleSheet`, et on peut faire varier dynamiquement le design d'un élément en fonction des props, du state ou de n'importe quelle autre variable. Cela ouvre ainsi d'infinies possibilités de mise en page et ouvre notamment la porte aux animations, que nous verrons dans un chapitre dédié.

VII. Auto-évaluation

A. Exercice final

Exercice

Exercice

Quel est le moteur sous-jacent utilisé par React Native pour le design ?

- ☐ Aurora
- ☐ YogaLayout
- ☐ EasyLayout
- ☐ LayoutEngine

Exercice

La méthode `StyleSheet.create` de l'API de React Native permet de...

- ☐ Optimiser les propriétés de style d'un objet
- ☐ Factoriser le style passé en paramètre pour le rendre plus condensé et performant
- ☐ Sécuriser les propriétés de style d'un objet et valider leur intégrité

Exercice

Il est possible d'utiliser des styles dynamiques dans React Native.

- ☐ Vrai
- ☐ Faux

Exercice

Quel est l'axe principal de Flex sur React Native ?

- ☐ L'axe horizontal
- ☐ L'axe vertical

Exercice

Laquelle de ces propriétés n'existe pas pour l'instruction `justifyContent` ?

- ☐ `flex-start`
- ☐ `space-after`

- ☐ space-between
- ☐ flex-end

Exercice

À quoi sert la propriété `flexGrow` ?

- ☐ Indiquer un facteur d'attribution de l'espace restant non occupé du parent
- ☐ Faire grossir un élément sur un facteur X de sa taille initiale
- ☐ Augmenter la taille de l'élément de facteur * 10 %

Exercice

On possède un parent de taille 100 px et deux enfants de taille 50 px : quel sera l'effet de l'instruction `flexGrow: 1` sur le premier et `flexGrow: 2` sur le deuxième ?

- ☐ Le deuxième sera deux fois plus gros que le premier
- ☐ Aucun effet
- ☐ Le parent va grandir pour atteindre la taille de 150 px

Exercice

On possède un parent de taille 100 px et deux enfants de taille 60 px : quel sera l'effet de l'instruction `flexShrink: 0` sur le premier et `flexShrink: 1` sur le deuxième ?

- ☐ Aucun effet, les deux éléments vont dépasser la taille du conteneur parent
- ☐ Le premier élément va ajuster sa taille automatiquement à 40 px
- ☐ Les deux éléments vont ajuster leur taille automatiquement à 50 px
- ☐ Le deuxième élément va ajuster sa taille automatiquement à 40 px

Exercice

Quelle est la différence entre l'instruction `justifyContent: "space-around"` et `justifyContent: "space-evenly"` ?

- ☐ Aucune, ce sont juste deux manières différentes d'écrire la même chose
- ☐ `space-around` applique quoi qu'il arrive la même taille avant et après lui, alors que `space-evenly` s'assure que l'espace entre le bord et/ou l'élément suivant/précédent soit exactement le même sur l'ensemble des éléments
- ☐ `space-around` applique un espacement uniquement autour de l'élément du milieu s'il y a trois éléments, alors que `space-evenly`, non

Exercice

Quelle instruction permet de centrer un élément sur la page d'une application qui a seulement un parent et un enfant ?

- ☐ Parent : rien
Enfant : `alignSelf: "center"`
- ☐ Parent : `justifyContent: "center"; alignItems: "center"`
Enfant : rien

- ☐ Parent: flex: 1; justifyContent: "center"; alignItems: "center"
Enfant: rien
- ☐ Parent: justifyContent: "center"; alignContent: "center"
Enfant: rien

B. Exercice : Défi

Reprenez l'exercice du chapitre 2 (*en indice ci-dessous*) et, cette fois-ci, créez un composant `CustomBox`, qui contiendra un champ de saisie permettant de donner la valeur de `flexGrow` de la boîte dans laquelle il se trouve.

Question

- Ce composant reçoit également une props `color`, qui est la couleur de la boîte.
- Quand la valeur change, l'interface doit donc s'adapter automatiquement sans recharger l'application. Pareil si l'on passe le téléphone en mode paysage.
- Attention, si le champ est vide, on considère que la valeur de `flexGrow` doit être de 1.
- Pour utiliser le champ de saisie, il faudra que votre composant maintienne un état et mette à jour la valeur.

Pour le champ de saisie, il faut suivre la documentation : <https://reactnative.dev/docs/textinput>.

Attention, il faut penser à bien convertir la valeur du champ `input` en nombre !

Indice :

```

1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     flex: 1, // Dit à la boîte de prendre 1/1 * 100 % (car seul enfant) de l'espace de l'axe
    principal en hauteur (height)
17     justifyContent: "center", // centre l'élément sur l'axe principal (vertical ici)
18     alignItems: "center" // et centre l'élément sur l'axe secondaire (horizontal)
19   },
20   firstBox: {
21     backgroundColor: 'green',
22     height: 70,
23     width: 100,
24   },
25   secondBox: {
26     backgroundColor: 'yellow',
27     height: 70,
28     width: 100,
29   },
30   thirdBox: {
31     backgroundColor: 'purple',
32     height: 70,
33     width: 100,
34   },

```

```
35 });
```

Contenus annexes

1. Flexbox

Objectif

- Positionner des éléments grâce à `flexbox`

Mise en situation

Pour faciliter le placement de blocs HTML, de nouvelles possibilités sont apparues au fil des années. Parmi celles-ci, `flexbox` nous permet de très facilement organiser notre contenu.

`Flexbox`, ou bien « modèle de boîte flexible », est un système de positionnement qui vise à faciliter et optimiser la disposition, l'alignement et la répartition entre les éléments.

Lorsqu'on utilise `flexbox`, on ne parlera plus des notions `inline`, `block` ou `float`.

Propriétés du conteneur flexible

Pour définir un élément conteneur en tant que conteneur flexible, il suffit de définir la propriété `display` à « `flex` ».

```
1 .container {
2   display: flex;
3 }
4
```

Les éléments enfants d'un conteneur flexible sont appelés « items ».

Flex-direction

Selon l'axe principal défini sur le conteneur grâce à la propriété `flex-direction`, les items seront affichés horizontalement (de gauche vers la droite ou vice versa), ou verticalement (de haut vers le bas ou vice versa).

```
1 .container {
2   flex-direction: row | row-reverse | column | column-reverse;
3 }
4
```

- `row` : de gauche vers la droite
- `row-reverse` : droite vers la gauche
- `column` : du haut vers le bas
- `column-reverse` : du bas vers le haut

La valeur par défaut de `flex-direction` est "`row`" : le comportement d'un conteneur `flex` est d'afficher tous ses enfants côte à côte sur une même ligne.

Flex-wrap

Par défaut, les items resteront sur une même ligne quitte à rétrécir s'ils n'ont pas la place de tous s'afficher. La propriété `flex-wrap` permet de modifier ce comportement.

```
1 .container {
2   flex-wrap: nowrap | wrap | wrap-reverse;
3 }
```

4

- nowrap : valeur par défaut, les items seront positionnés sur une même ligne même si cela implique de les rétrécir.
- wrap : si les éléments n'ont pas la place de s'afficher en largeur sans être rétrécis, les items qui débordent s'afficheront sur la ligne du dessous.
- wrap-reverse : même comportement qu'avec wrap, mais les items s'afficheront cette fois du bas vers le haut.

Exemple

```

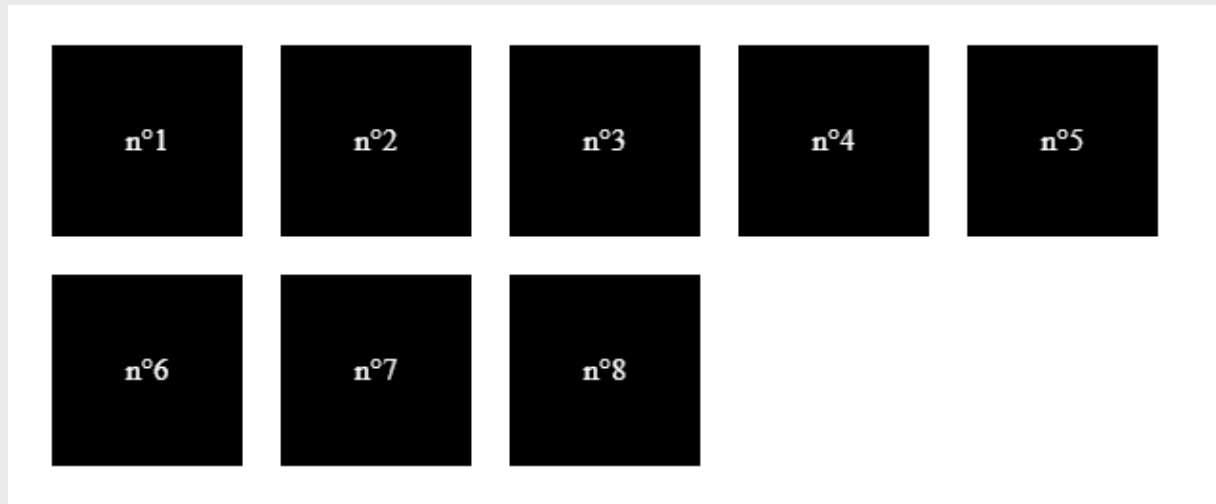
1 <ul class="container">
2   <li class="item">n°1</li>
3   <li class="item">n°2</li>
4   <li class="item">n°3</li>
5   <li class="item">n°4</li>
6   <li class="item">n°5</li>
7   <li class="item">n°6</li>
8   <li class="item">n°7</li>
9   <li class="item">n°8</li>
10 </ul>
11
12 .container {
13   list-style: none;
14   display: flex;
15   flex-wrap: nowrap;
16 }
17
18 .item {
19   background: black;
20   width: 100px;
21   height: 100px;
22   margin: 10px;
23   color: white;
24   text-align: center;
25   line-height: 100px;
26 }
```

Résultat: nowrap



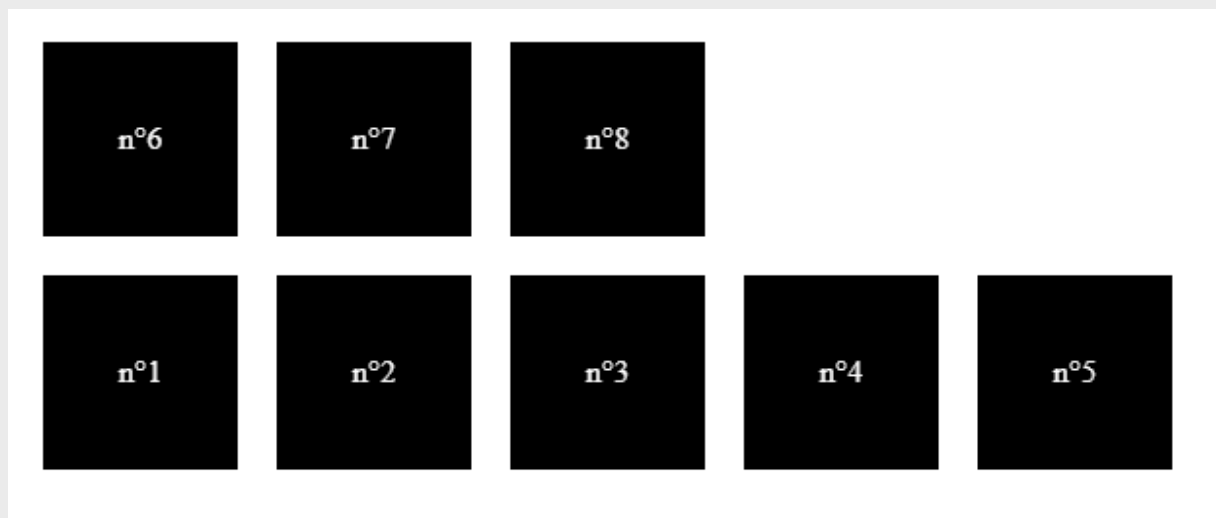
On peut voir que les éléments sont carrés (100 px par 100 px) dans leur définition, mais la fenêtre est trop petite pour permettre d'afficher les 8 enfants à 100 px de large : `flex` va automatiquement les redimensionner pour les afficher sur la même ligne.

Résultat : `wrap`



Avec `wrap`, les éléments retrouvent leur taille comme nous l'avons définie, et les trois éléments qui ne peuvent pas s'afficher sur la première ligne sont repoussés en dessous.

Résultat : `wrap-reverse`



Remarque

Il existe une propriété combinant les deux propriétés vues ci-dessus (`flex-direction` et `flex-wrap`) : `flex-flow`.

```
1 flex-flow: row nowrap
```

Justify-content

En suivant l'axe principal défini par `flex-direction`, la propriété `justify-content` permet de gérer l'alignement des items sur le conteneur flexible sur cet axe principal.

```
1 .container {
```

```

2   justify-content: flex-start | flex-end | center | space-between | space-around | space-
3   evenly;
4 }

```

- `flex-start` : les items seront alignés par rapport au début de l'axe principal (à gauche si l'axe est horizontal, en haut s'il est vertical, à droite si l'axe est horizontal *reverse* et en bas s'il est vertical *reverse*)
- `flex-end` : les items seront alignés par rapport à la fin de l'axe principal (contraire de `flex-start`)
- `center` : ils seront alignés au centre sur l'axe principal
- `space-between` : ils sont répartis sur l'axe principal : le premier item sera placé au début de la ligne et le dernier à la fin
- `space-around` : les items sont répartis le long de l'axe principal avec un espace égal autour d'eux (mais, attention, il n'y aura pas le même espace avec les deux bords extrêmes)
- `space-evenly` : les items sont répartis le long de l'axe principal avec un espace égal entre eux (même espace avec les deux bords extrêmes)

À noter que `flex` utilise les notions de "début" et de "fin" d'un axe : on ne parle pas de "gauche" ou "droite". Certains langages sont lus de droite à gauche comme l'arabe, et de plus en plus de dispositions sont prises par les langages web et les navigateurs pour écrire du code facilement compatible avec les langages LTR (*Left To Right*) et RTL (*Right To Left*).

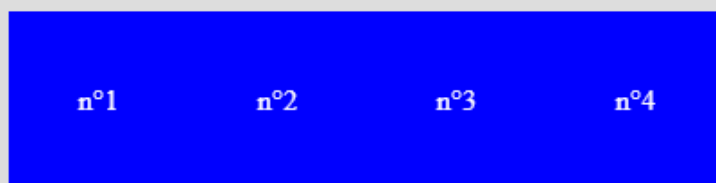
Exemple

```

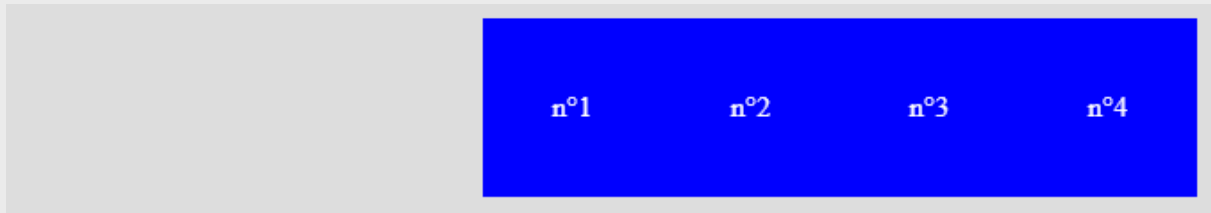
1 <ul class="container">
2   <li class="item">1</li>
3   <li class="item">2</li>
4   <li class="item">3</li>
5   <li class="item">4</li>
6   <li class="item">5</li>
7   <li class="item">6</li>
8   <li class="item">7</li>
9   <li class="item">8</li>
10 </ul>
11
12 .container {
13   list-style: none;
14   display: flex;
15   justify-content: flex-start;
16 }

```

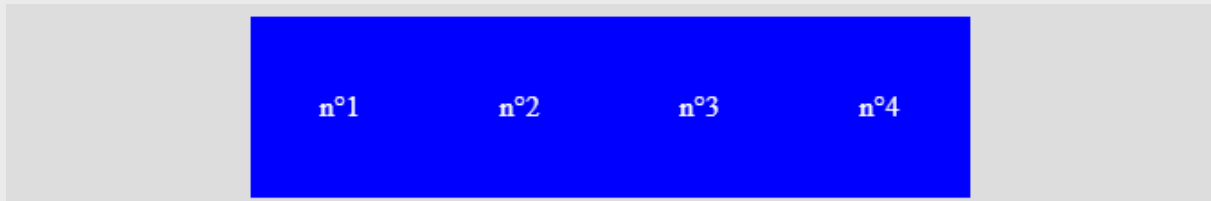
Résultat : `flex-start`



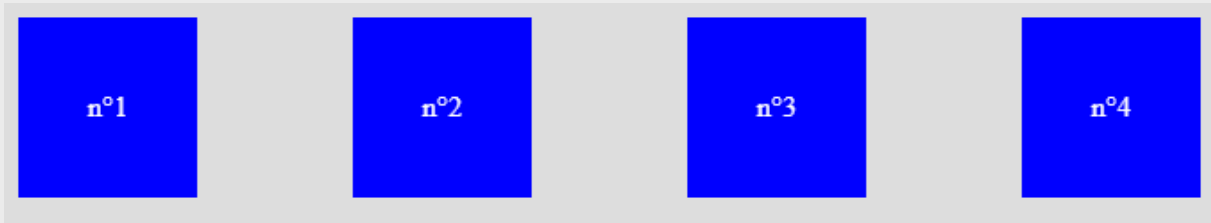
Résultat : flex-end



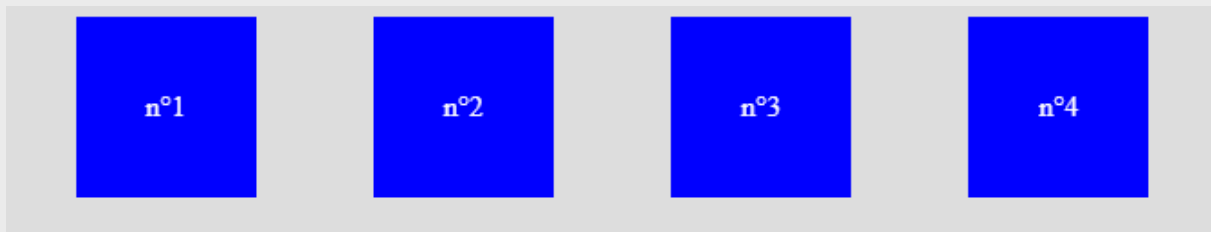
Résultat : center



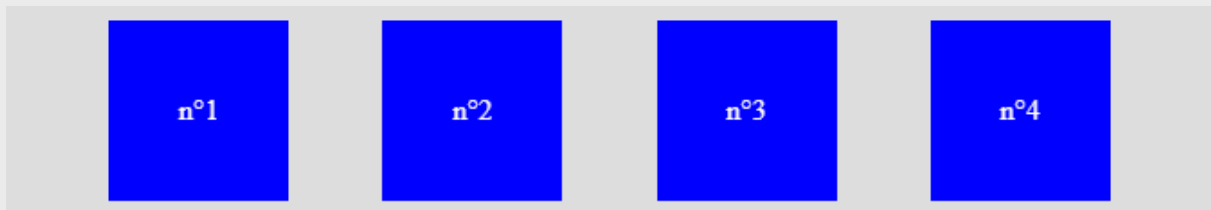
Résultat : space-between



Résultat : space-around



Résultat : space-evenly



Align-items

Nous venons de voir l'alignement des items le long de l'axe principal, il va être intéressant de voir à présent l'alignement le long d'une ligne sur le second axe (axe transversal).

Pour faire cela, nous allons utiliser la propriété `align-items`.

```
1.container {
```

```

2   align-items: stretch | flex-start | flex-end | center | baseline
3 }
4

```

- `stretch` : les items seront étirés pour prendre toute la place du conteneur flexible.
- `flex-start` : les items seront placés au début de l'axe transversal.
- `flex-end` : les items seront placés à la fin de l'axe transversal.
- `center` : les items seront alignés au centre sur l'axe transversal.
- `baseline` : les items seront alignés en fonction de leur ligne de base.

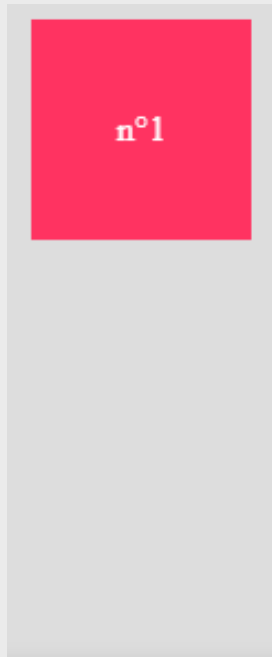
Exemple

```

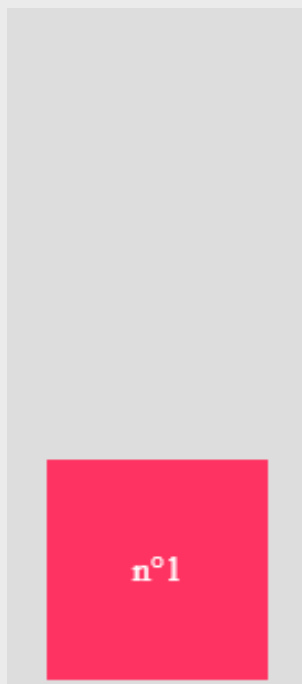
1 <ul class="container">
2   <li class="item">n°1</li>
3 </ul>
4
1 body{
2   background: #ddd;
3   height:200px;
4 }
5 .container {
6   list-style: none;
7   display: flex;
8   justify-content: center;
9   margin:0;
10  padding:0;
11  height:300px;
12  align-items: baseline;
13  flex-wrap: wrap;
14 }
15
16 .item {
17   background: #FF3361;
18   width: 100px;
19   height: 100px;
20   color: white;
21   text-align: center;
22   line-height: 100px;
23 }
24 }
25 .item2 {
26   font-size:2em;
27   background: #FF3361;
28 }
29

```

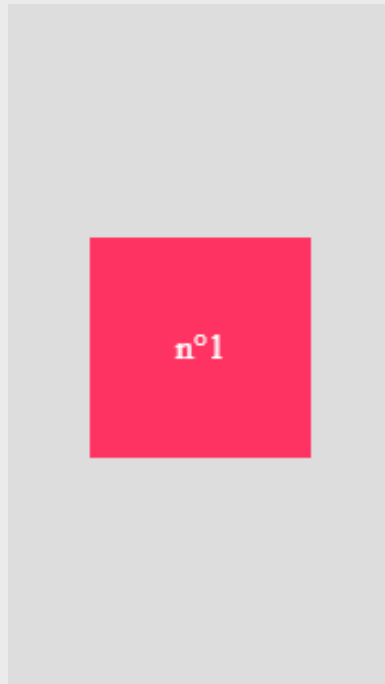
Résultat : `flex-start`



Résultat : `flex-end`



Résultat : center



Résultat : baseline

```
1 <ul class="container">
2   <li class="item">n°1</li>
3   <li class="item item2">n°2</li>
4 </ul>
5
```



Résultat : stretch

```
1 <ul class="container">
2   <li class="item">1<br>2</li>
3   <li class="item">3</li>
4   <li class="item">4<br>5</li>
5   <li class="item">6</li>
6 </ul>
7
1 .container {
```

```

2   padding: 0;
3   margin: 0;
4   list-style: none;
5   display: flex;
6 }
7
8 li {
9     background: #FF3361;
10 }
11
12 .item {
13     padding: 5px;
14     width: 100px;
15     margin: 5px;
16     line-height: 50px;
17     color: white;
18     font-weight: bold;
19     font-size: 2em;
20     text-align: center;
21 }
22

```



Align-content

Comme nous venons de le voir, la propriété `align-items` permet d'aligner les items d'une ligne sur l'axe transversal. Il existe également une propriété permettant de gérer l'alignement des lignes le long de l'axe transversal. Les valeurs possibles sont les mêmes que celles de la propriété `justify-content`.

Exemple

```

1 <ul class="container">
2   <li class="item">1<br>1 bis</li>
3   <li class="item">2</li>
4   <li class="item">3</li>
5   <li class="item">4</li>
6   <li class="item">5</li>
7   <li class="item">6</li>
8   <li class="item">7</li>
9 </ul>
10
11 .container {

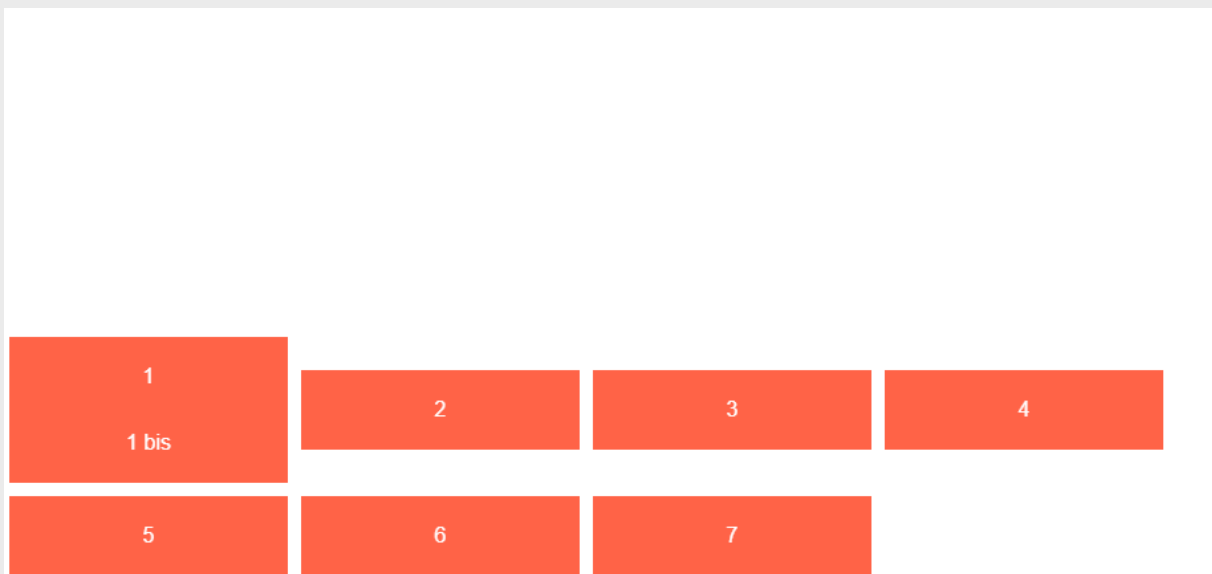
```

```

2   padding: 0;
3   margin: 0;
4   list-style: none;
5   display: flex;
6   flex-wrap: wrap;
7   align-items: center;
8   align-content: flex-end;
9   height: 100vh;
10 }
11
12 .item {
13   background: orange;
14   padding: 5px;
15   width: 200px;
16   margin: 5px;
17   line-height: 50px;
18   color: white;
19   text-align: center;
20 }

```

Résultat :



Propriétés des items

Nous avons vu comment agir sur le contenant, mais nous pouvons aussi agir sur le contenu grâce à une autre propriété.

Définition **Order**

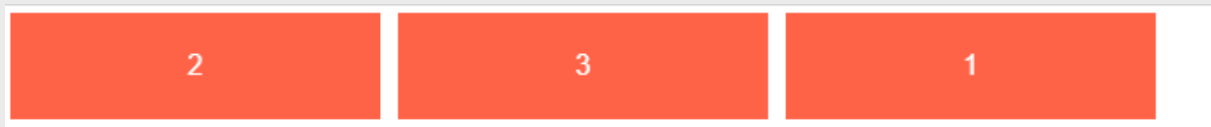
Au sein d'un conteneur flexible, les items sont affichés par défaut dans l'ordre d'écriture dans le fichier HTML. Il existe un moyen de modifier ce comportement en CSS grâce à la propriété **order**.

Exemple

```
1 <ul class="container">
2   <li class="item one">1</li>
3   <li class="item two">2</li>
4   <li class="item three">3</li>
5 </ul>
6
```

```
1 .container {
2   padding: 0;
3   margin: 0;
4   list-style: none;
5   display: flex;
6   flex-wrap: wrap;
7   align-items: center;
8   align-content: flex-start;
9   height: 100vh;
10 }
11
12
13 .item {
14   background: orange;
15   padding: 5px;
16   width: 200px;
17   margin: 5px;
18   line-height: 50px;
19   color: white;
20   text-align: center;
21 }
22
23 .one {
24   order: 2;
25 }
26
27 .two {
28   order: 0;
29 }
30
31 .three {
32   order: 1;
33 }
34
```

Résultat :



Taille d'un enfant

La propriété **flex** sur un enfant d'un conteneur flexible permet de régler la taille de l'enfant de manière dynamique, un peu à la manière de l'unité %.

Cette propriété est le raccourci de trois propriétés qu'il est possible de définir unitairement, dans l'ordre :

- **flex-grow** : permet de définir la capacité de l'élément à s'élargir au besoin pour remplir le conteneur flexible. Accepte une valeur numérique positive.
- **flex-shrink** : permet de définir la capacité de l'élément à rétrécir au besoin pour laisser la place à d'autres éléments d'être affichés dans le conteneur flexible. Accepte une valeur numérique positive.
- **flex-basis** : permet d'indiquer une taille par défaut pour l'élément avant que l'espace restant dans le conteneur flexible ne soit redistribué, la valeur auto indiquera au navigateur de prendre la valeur de `width` par défaut. Accepte une taille exprimée dans n'importe quelle unité supportée par `width` ou `height` (px, pt, %, em, ...).

Soit un conteneur flexible de 1000 px de largeur : imaginons deux enfants, dont la taille souhaitée est de respectivement 1/3 et 2/3 du conteneur flexible.

```

1 .parent {
2   display: flex;
3   width: 1000px;
4 }
5
6 .enfant1 {
7   flex: 1;
8 }
9
10 .enfant2 {
11   flex: 2;
12 }
13
14 // enfant1 et enfant2 vont automatiquement s'élargir pour couvrir l'espace disponible dans le
   conteneur flexible ".parent"
15 // enfant2 prendra deux fois plus de place que enfant1, ce qui revient à leur donner 1/3 et
   2/3 de l'espace disponible.
```



Pour répartir équitablement la place disponible, il suffit de mettre la même valeur pour les deux enfants.

```

1 .parent {
2   display: flex;
3   width: 1000px;
4 }
5
6 .enfant1 {
7   flex: 1;
8 }
9
10 .enfant2 {
11   flex: 1;
12 }
13
14 // Les deux enfants se partageront l'espace disponible
```




Flex-shrink permet de gérer la distribution de l'espace dans un conteneur flexible s'il n'y a pas assez de place pour afficher tous les éléments sans les déformer.

10 enfants de 200 px de large sont définis pour un conteneur de 1000 px de large, `flex-wrap` n'est pas activé, donc `flexbox` va conserver tous les éléments sur la même ligne. Les 10 enfants ont alternativement une valeur de `flex-shrink` de 1 et 2.

```
1 .enfant1 {
2   flex-shrink: 1;
3   background: #0984e3;
4 }
5
6 .enfant2 {
7   flex-shrink: 2;
8   background: #00b894;
9 }
10
11 .enfant3 {
12   flex-shrink: 1;
13   background: #a29bfe;
14 }
15
16 .enfant4 {
17   flex-shrink: 2;
18   background: #fdbc6e;
19 }
20
21 .enfant5 {
22   flex-shrink: 1;
23   background: #00cec9;
24 }
25
26 .enfant6 {
27   flex-shrink: 2;
28   background: #e17055;
29 }
30
31 .enfant7 {
32   flex-shrink: 1;
33   background: #e84393;
34 }
35
36 .enfant8 {
37   flex-shrink: 2;
38   background: #2d3436;
39 }
40
41 .enfant9 {
42   flex-shrink: 1;
43   background: #1e3799;
44 }
45
46 .enfant10 {
```

```
47 flex-shrink: 2;
48 background: #f6b93b;
49 }
```

Le résultat montre qu'un élément sur deux s'est plus déformé pour laisser la place aux éléments pour lesquels a été indiqué le `flex-shrink` le plus faible.



Syntaxe À retenir

Pour déclarer un contenu flexible, il faut ajouter la règle CSS `display: flex`.

Plusieurs options pour manipuler l'affichage du contenu sont disponibles :

- `flex-direction` : pour définir la façon dont les éléments sont placés
- `flex-wrap` : pour définir si les éléments peuvent être placés à la ligne ou non
- `justify-content` : pour définir l'espace entre les éléments
- `align-items` : pour définir l'alignement des éléments
- `align-content` : pour définir l'espace entre et autour des éléments
- `order` : pour définir l'ordre d'affichage des éléments

Remarque Flexbox vs float

Nous venons de voir deux manières de gérer le *layout* de la page pour afficher des éléments sur une même ligne. `Flexbox` est plus récent et permet de positionner plus finement les éléments sur la page, il est donc à préconiser par rapport à `float`, sauf dans certains scénarii particuliers où `float` est adapté, comme venir faire s'enrouler un texte autour d'une boîte. `Flexbox` est un peu plus complexe à prendre en main, mais c'est un outil très puissant qui permet de couvrir presque tous les besoins de *layout* d'une page web.

Complément

Documentation MDN sur les Flexbox¹

Solutions des exercices

¹ https://developer.mozilla.org/fr/docs/Web/CSS/CSS_Box_Alignment/Alignement_bo%C3%A0etes_disposition_Flexbox

Exercice p. Solution n°1

```

1 import * as React from 'react';
2 import { Button, Text, View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   const [bgColor, setBgColor] = React.useState('yellow');
6
7   React.useEffect(() => {
8     // On crée un code qui s'exécute à interval régulier
9     const interval = setInterval(() => {
10       setBgColor((previous) => (previous === 'yellow' ? 'green' : 'yellow'));
11     }, 3000);
12
13     // Et on n'oublie pas de libérer la mémoire pour éviter les fuites de mémoire quand le
    composant est démonté
14     return () => {
15       clearInterval(interval);
16     };
17   }, []);
18
19   return (
20     <View
21       style={[
22         styles.container,
23         {
24           backgroundColor: bgColor,
25         },
26       ]}>
27       <Text style={styles.firstText}>Bienvenue</Text>
28       <Text style={styles.secondText}>dans ce</Text>
29       <Text style={styles.thirdText}>Tutoriel</Text>
30     </View>
31   );
32 }
33
34 const styles = StyleSheet.create({
35   container: {
36     flex: 1,
37     justifyContent: 'center',
38     alignItems: 'center',
39   },
40   firstText: {
41     color: 'white',
42     fontWeight: 'bold',
43   },
44   secondText: {
45     color: 'blue',
46     fontWeight: 'bold',
47   },
48   thirdText: {
49     color: 'red',
50     fontWeight: 'bold',
51   },
52 });

```

[cf. video-style-1]

Exercice p. Solution n°2

```

1 import * as React from 'react';
2 import { View, StyleSheet } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <View style={styles.firstBox} />
8       <View style={styles.secondBox} />
9       <View style={styles.thirdBox} />
10    </View>
11  );
12}
13
14 const styles = StyleSheet.create({
15   container: {
16     flexDirection: 'row', // On change l'axe principal
17     flex: 1, // Dit à la boîte de prendre 1/1 * 100 % (car seul enfant) de l'espace de l'axe
18     principal (horizontal donc width)
19   },
20   firstBox: {
21     backgroundColor: 'green',
22     height: 50,
23     flex: 2, // On donne la valeur 2 à flexGrow
24   },
25   secondBox: {
26     backgroundColor: 'yellow',
27     height: 50,
28     flexGrow: 3, // On peut également l'écrire comme cela
29   },
30   thirdBox: {
31     backgroundColor: 'purple',
32     height: 50,
33     flex: 1,
34   },
35 });

```


Exercice p. 18 Solution n°3

Exercice

Quel est le moteur sous-jacent utilisé par React Native pour le design ?

- ☐ Aurora
- ☒ YogaLayout
- ☐ EasyLayout
- ☐ LayoutEngine

 La bonne réponse est la librairie YogaLayout. C'est d'ailleurs probablement la seule qui existe aujourd'hui.

Exercice

La méthode `StyleSheet.create` de l'API de React Native permet de...

- ☐ Optimiser les propriétés de style d'un objet
- ☐ Factoriser le style passé en paramètre pour le rendre plus condensé et performant
- ☒ Sécuriser les propriétés de style d'un objet et valider leur intégrité

 `StyleSheet.create` permet de certifier que les propriétés passées en paramètres via un objet utilisent des valeurs qui existent. Cela permet également l'auto-complétion via son IDE.

Exercice

Il est possible d'utiliser des styles dynamiques dans React Native.

- ☒ Vrai
- ☐ Faux

 On peut effectivement utiliser des styles dynamiques et faire varier ces propriétés au moment du *render* en utilisant les props ou n'importe quelle autre variable.

Exercice

Quel est l'axe principal de Flex sur React Native ?

- ☐ L'axe horizontal
- ☒ L'axe vertical

 Contrairement à son homologue web, React Native utilise par défaut l'axe vertical pour simplifier le développement des applications mobiles, avec des petits écrans souvent plus hauts que larges.

Exercice

Laquelle de ces propriétés n'existe pas pour l'instruction `justifyContent` ?

- ☐ flex-start
- ☒ space-after
- ☐ space-between
- ☐ flex-end

 La propriété `space-after` est la seule qui n'existe pas.

Exercice

À quoi sert la propriété `flexGrow` ?

- ☒ Indiquer un facteur d'attribution de l'espace restant non occupé du parent
- ☐ Faire grossir un élément sur un facteur X de sa taille initiale
- ☐ Augmenter la taille de l'élément de facteur * 10 %
-  Lorsque l'espace du parent n'est pas comblé par les enfants, il est possible de spécifier à ces enfants une répartition de cet espace entre eux afin de les faire grandir.

Exercice

On possède un parent de taille 100 px et deux enfants de taille 50 px : quel sera l'effet de l'instruction `flexGrow: 1` sur le premier et `flexGrow: 2` sur le deuxième ?

- ☐ Le deuxième sera deux fois plus gros que le premier
- ☒ Aucun effet
- ☐ Le parent va grandir pour atteindre la taille de 150 px
-  Étant donné la taille des éléments et la taille du parent, l'espace restant à distribuer n'existe pas et vaut 0. Cela n'a donc aucun effet.

Exercice

On possède un parent de taille 100 px et deux enfants de taille 60 px : quel sera l'effet de l'instruction `flexShrink: 0` sur le premier et `flexShrink: 1` sur le deuxième ?

- ☐ Aucun effet, les deux éléments vont dépasser la taille du conteneur parent
- ☐ Le premier élément va ajuster sa taille automatiquement à 40 px
- ☐ Les deux éléments vont ajuster leur taille automatiquement à 50 px
- ☒ Le deuxième élément va ajuster sa taille automatiquement à 40 px
-  L'espace du parent n'étant pas suffisant pour contenir ses enfants, en utilisant `flexShrink`, on donne un facteur de réduction de taille aux éléments. Le deuxième élément va donc s'adapter pour atteindre la taille du parent.

Exercice

Quelle est la différence entre l'instruction `justifyContent: "space-around"` et `justifyContent: "space-evenly"` ?

- ☐ Aucune, ce sont juste deux manières différentes d'écrire la même chose
- ☒ `space-around` applique quoi qu'il arrive la même taille avant et après lui, alors que `space-evenly` s'assure que l'espace entre le bord et/ou l'élément suivant/précédent soit exactement le même sur l'ensemble des éléments
- ☐ `space-around` applique un espacement uniquement autour de l'élément du milieu s'il y a trois éléments, alors que `space-evenly`, non
-  Le schéma ci-dessous montre que la bonne réponse est la deuxième.

**flex-start****flex-end****center****space-between****space-around****space-evenly**

Exercice

Quelle instruction permet de centrer un élément sur la page d'une application qui a seulement un parent et un enfant ?

- ☐ Parent:rien
Enfant:alignSelf: "center"
- ☐ Parent:justifyContent: "center"; alignItems: "center"
Enfant:rien
- ☒ Parent:flex: 1; justifyContent: "center"; alignItems: "center"
Enfant:rien

- Parent:justifyContent: "center"; alignContent: "center"
 Enfant: rien

Q Si le parent ne possède pas flex: 1, il ne va pas lui-même remplir l'intégralité de l'écran et se contenter de la taille de son enfant. Ainsi, on ne pourra pas centrer.

Une fois qu'on rajoute flex: 1, on a bien une boîte qui fait la taille de l'application et on peut donc la centrer correctement avec nos deux propriétés.

Exercice p. Solution n°4

```

1 import * as React from 'react';
2 import { View, StyleSheet, TextInput } from 'react-native';
3
4 export default function App() {
5   return (
6     <View style={styles.container}>
7       <CustomBox color="blue" />
8       <CustomBox color="gray" />
9       <CustomBox color="red" />
10    </View>
11  );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     flexDirection: 'row', // On change l'axe principal
17     flex: 1, // Dit à la boîte de prendre 1/1 * 100 % (car seul enfant) de l'espace de l'axe
18     // principal (horizontal donc width)
19   },
20 });
21
22 function CustomBox(props) {
23   const [flexSize, setFlexSize] = React.useState(1);
24
25   return (
26     <View
27       style={{
28         flex: flexSize,
29         backgroundColor: props.color,
30       }}>
31       <TextInput
32         onChangeText={(value) => setFlexSize(Number(value !== '' ? value : 1))}
33         value={flexSize}
34       />
35     </View>
36   );
37 }

```